rittman analytics

WIRE FRAMEWORK · WORKING IN A LIVE REPO

Picking up an in-flight engagement.

Validate, diagnose, fix, sign off — without breaking what's already there.

Developer Onboarding | Rittman Analytics

V3.7.0

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

WHEN THIS PATTERN APPLIES

Most days of a Wire engagement look like this.

01 **Joining a project you didn't start.** The repo exists, `.wire/` is populated, dbt models are in the warehouse — and you need to be productive in an hour.

02 **CI flagged a validation failure** and you need to understand which convention was broken before guessing a fix.

03 **The sponsor asked for a small change** — a new column, a renamed metric, a missing test — on a release that's otherwise complete.

04 **A teammate's PR is sitting in review** and you need to figure out what they actually changed.

T: +44 (0)208 058 5951 W: rittmananalytics.com E: info@rittmananalytics.com

rittman analytics

ONE LOOP, TWO FLAVOURS

Fixing bugs and adding features are the same workflow.

SCENARIO	THE HEADLINE COMMAND SEQUENCE
Fix a validation failure	`/wire:<artifact>-validate` flags an issue → focused `claude -p` fix → re-validate
Add a feature	`/wire:<artifact>-generate` extends the artifact → `-validate` checks conventions → focused `claude -p` cleanup → re-validate

Both go through the same gates: **status → validate → fix or extend → re-validate → build → review.** Don't think of them as separate workflows — they're the same six-step loop with different content in the middle.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

THE FLOW AT A GLANCE

Six gates. Two need judgement; four are mechanical.

LLM-driven — judgement required (validate, fix) Mechanical — start, status, dbt build, review

Onboard in two commands
`start` + `status` and you know exactly where things stand.

One validate, one fix, one re-validate
That's the core loop — everything else is setup and sign-off.

dbt build is the runtime check
Wire's conventions and dbt's compilation are separate guardrails.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

STEP 1 OF 8 · ONBOARD

1 Onboard with `/wire:start`

COMMAND	`/wire:start`
WHAT IT DOES	Reads `.wire/engagement/context.md`, lists active releases, surfaces the most recent session note and the next-step suggestion.
HITL GATE	None — read-only.

● RUN IT FIRST

The only command that auto-loads engagement context.

Run it the moment you walk into a repo — it stops you poking around blind.

TYPICAL OUTPUT

acme-coffee-data-platform

```
/wire:start

Engagement: Acme Coffee Roasters – marketing_mart_extension

Active releases:
  01-marketing-mart (dbt_development)
    current_phase: dbt_validate · blocked

Suggested next:
  /wire:status releases/01-marketing-mart
```

rittman analytics

STEP 2 OF 8 · READ

2 See what's failing with `/wire:status`

COMMAND	`/wire:status releases/01-marketing-mart`
INPUTS	`releases/<folder>/status.md`
SHOWS	Every artifact's generate / validate / review state, plus session history and the last failing validation summary.
HITL GATE	None — read-only.

SOURCE OF TRUTH

`status.md` is the project's source of truth.

Wire writes to it after every generate / validate / review. Trust the repo, trust status.md.

WHAT STATUS SHOWS

releases/01-marketing-mart/status.md

```
artifact          generate  validate  review
──────────────────────────────────────────────
requirements      ✓         ✓         ✓
conceptual_model  ✓         ✓         ✓
data_model        ✓         ✓         ✓
dbt               ✓         ✗ fail    —

dbt.validate: failing
  warehouse/schema.yml missing FK
  relationships test
```


T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

STEP 3 OF 8 · AI CHECK

3 Run validate, see Wire's verdict

COMMAND	`/wire:dbt-validate releases/01-marketing-mart`
PRODUCES	A severity-rated report covering naming, testing coverage, documentation, model config, and convention compliance.
HITL GATE	None — AI-driven report.

● DETAILED & ACTIONABLE

Validate reads the dbt project and the convention library together.

You shouldn't have to ask a human "what does this mean" after a validate run.

FOR DEMO 2

CRITICAL **warehouse/schema.yml incomplete**

`fct_orders.customer_fk` is undocumented and has no relationships test against `dim_customer.customer_pk`.

> **Wire convention:** every fact-table FK must have a documented `relationships` test pointing at the referenced dimension PK.

T: +44 (0)208 058 5951 W: rittmananalytics.com E: info@rittmananalytics.com

rittman analytics

STEP 4 OF 8 · DIAGNOSE

4 Diagnose using the convention library

SOURCE	`wire/skills/dbt-development/SKILL.md` — the convention source of truth.
WHY IT MATTERS	Missing the relationships test means broken referential integrity could ship to production undetected.
NOTE	Skills auto-activate during ad-hoc dbt work too — they don't only fire inside `/wire:*` commands.

● READ THE "WHY"

Read the convention skill before fixing.

~200 lines that tell you *why* the convention exists, not just what it is. The "why" makes the fix obvious.

WAREHOUSE-LAYER TESTING CONVENTIONS

SKILL.MD **dbt-development**

- ✓ Every **PK** has `unique` + `not_null` tests.
- ✓ Every **FK** has `not_null` + `relationships(to:, field:)` test.
- ✓ Every **column** has a `description` in `schema.yml`.

rittman analytics

STEP 5 OF 8 · AI EDIT

5 Focused fix via `claude -p`

SCOPE	"Make ONLY that change" keeps Claude from refactoring nearby code.
ANCHOR	Pin the change to a file + line region + the rule being enforced.
FEATURES TOO	Same pattern — swap "add the missing FK entry" for "add an `order_value_tier` column derived from `total_price` thresholds".

THE FIX PROMPT — ONE PARAGRAPH

```
>_ CLAUDE -P

In dbt/models/warehouse/schema.yml, the fct_orders model only
documents order_pk. Add a customer_fk column entry with: (a) a
description noting it is the foreign key to dim_customer, (b) a
not_null test, and (c) a relationships test against
ref("dim_customer") on customer_pk. Make ONLY that addition;
leave every other line unchanged.
```

NARROW WINS

The narrower the prompt, the more reliable the edit.

A vague "fix the validation issues" rewrites half the project. A paragraph like this changes seven lines.

rittman analytics

STEP 6 OF 8 · AI CHECK

6 Re-validate

COMMAND	`/wire:dbt-validate releases/01-marketing-mart`
EXPECTED	PASS on the previously failing check; pre-existing PASS items still PASS.
WATCH FOR	Did the LLM introduce a different convention violation while fixing the original one? Wire's validate catches it.

DON'T SKIP IT

About one in five LLM fixes introduces a new issue elsewhere.

Most often a missing test on a column that was just added. Re-validate catches it.

RE-VALIDATE RESULT

PASS **dbt-validate · all checks green**

- ✓ `fct_orders.customer_fk` documented with a description.
- ✓ `not_null` test present on `customer_fk`.
- ✓ `relationships` test → `dim_customer.customer_pk`.
- ✓ No new violations introduced elsewhere.

rittman analytics

STEP 7 OF 8 · RUNTIME

7 Confirm runtime with `dbt build`

COMMAND	`dbt deps && dbt build`
TWO GUARDRAILS	Wire validates conventions; dbt validates that the SQL actually works. Both have to be green.
DEMO 2 RESULT	**PASS=28, ERROR=0** — 2 seeds + 5 models + 21 tests, including the new FK relationships test.

THE SAFETY NET

validate green + build green = both guardrails clear.

Validate passes but build fails → a Wire convention bug. Build passes but validate fails → dbt-but-not-Wire-compliant.

DBT BUILD

acme-coffee-data-platform

```
dbt deps && dbt build

Running with dbt=1.8 · 28 models, seeds & tests

PASS seed acme.raw_orders ................ OK
PASS model warehouse.fct_orders .......... OK
PASS model warehouse.dim_customer ........ OK
PASS test relationships_fct_orders_
     customer_fk__customer_pk__ref_
     dim_customer_ ..................... OK

Done. PASS=28  WARN=0  ERROR=0
```

rittman analytics

STEP 8 OF 8 · SIGN OFF

8 Review and sign-off

COMMAND	`/wire:dbt-review releases/01-marketing-mart`
WHAT IT DOES	Surfaces a review summary, records reviewer + outcome in `status.md`, appends an entry to `execution_log.md`.
HITL GATE	Real engagement → sponsor or peer review. In the demo we auto-approve and narrate the simulation.

● THE AUDIT TRAIL

`status.md` + `execution_log.md` are the audit trail.

"Who approved the FK rename in week 4?" — the answer is in version control.

FOR DEMO 2

APPROVED dbt.review

Status updated: `dbt.review: approved`, `reviewed_by: Demo Operator`, `reviewed_date: 2026-05-31`.

> The release is now ready to proceed to **deployment preparation**.

rittman analytics

SAME LOOP, DIFFERENT CONTENT

To add a feature instead of fixing a bug, only step 5 changes.

STEP	FIX WORKFLOW	FEATURE WORKFLOW
1–3	`/wire:start` ⟶ status ⟶ validate	`/wire:start` ⟶ status ⟶ validate *(sanity)*
4	Read the convention you violated	Read the convention you must satisfy
5	`claude -p "fix X by doing Y"`	`claude -p "add column X to fct_orders, with these tests…"` OR `/wire:dbt-generate` with feature-specific scope
6–8	Re-validate ⟶ build ⟶ review	Re-validate ⟶ build ⟶ review

Adding a feature is just a fix for "the artifact doesn't have this thing yet." The discipline of **validate ⟶ fix ⟶ re-validate** is what stops feature work from breaking what's already there.

T: +44 (0)208 058 5951 W: rittmananalytics.com E: info@rittmananalytics.com

rittman analytics

GETTING STARTED

Three commands before you change anything.

```
# Inside the client repo
cd ~/clients/acme-coffee-data-platform
/wire:start

# Look at what's open
/wire:status

# Validate before changing anything
/wire:dbt-validate releases/<release-folder>
```

For features spanning multiple artifacts (e.g. "add Klaviyo as a source"), use `/wire:dbt-generate` with the new scope rather than hand-prompting — Wire sequences the staging / integration / warehouse work for you.

LEARN IT FIRST

Run Demo 2 once before your first day on a live engagement.

The end-to-end demo lives in `wire-demos/demo2-fix-an-issue/` — it runs in ~5 minutes against a planted convention violation. The full loop in five minutes against a known fault is the fastest way to internalise the rhythm.

Convention library: `wire/skills/dbt-development/SKILL.md` · skills index: `wire/skills/README.md`.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com